

MetaPrep: Data preparation pipelines recommendation via meta-learning

Fernando Rezende Zagatti^{*†}, Lucas Cardoso Silva^{*†}, Lucas Nildaimon dos Santos Silva^{*†},
Bruno Silva Sette^{*†}, Helena de Medeiros Caseli[†], Daniel Lucrédio[†] and Diego Furtado Silva[†]

^{*}Americanas S.A. - Digital Lab, São Paulo, SP, Brazil

[†]Federal University of São Carlos - Department of Computing, SP, Brazil

Email: fernando.zagatti@estudante.ufscar.br, lucascardoso@estudante.ufscar.br, lucas.silva@estudante.ufscar.br,
bruno.sette@ufscar.br, helenacaseli@ufscar.br, daniel.lucradio@ufscar.br, diegofs@ufscar.br

Abstract—Data preparation is a mandatory phase in the machine learning pipeline. The goal of data preparation is to convert noisy and disordered data into refined data that can be used by the algorithms. However, data preparation is time-consuming and requires specialized knowledge about the data and algorithms. Therefore, automating data preparation is essential to decrease the effort made by data scientists to develop satisfactory models. Despite its relevance, current AutoML platforms disregard or make simple hardcoded data preparation pipelines. Trying to fill this gap, we present a meta-learning-based recommendation system for data preparation. Our system recommends five pipelines, ranked by their relevance, making it useful for users with varying degrees of experience. Using the top-1 pipeline we demonstrated that our proposal allows a better performance of an AutoML system. Furthermore, the accuracy rates of our method were comparable to those achieved by a reinforcement-learning-based algorithm with the same goal, but it was up to two orders of magnitude faster. Moreover, we tested our method in a real-world application and evaluated its benefits and limitations in this scenario.

Index Terms—Meta-learning, data preparation, automated machine learning

I. INTRODUCTION

Given that real data¹ is inherently noisy and varied, several preprocessing procedures are required during the development of a Machine Learning (ML) model. Due to missing data, contradictions, and incorrect information present in the real data, it is impossible to learn good ML models from this data without any preprocessing [1].

There are some tools that aim to perform data cleaning automatically, such as Auto-Sklearn, which performs a preprocessing pre-defined by the user, and Learn2Clean, which sought to automate data preparation via reinforcement learning [2]. However, to the best of our knowledge, none of them is able to handle categorical data and, because they are iterative processes, they end up being time-consuming and computationally expensive.

Automated machine learning (AutoML) has risen with the purpose of automatically define the best ML algorithms and configurations, as well as optimizing the performance of the generated models [3]. However, current AutoML platforms usually perform little or no data preparation, acting only on the feature engineering and further steps [4].

¹Real data is data produced and processed by companies.

In this work we follow the hypothesis that meta-learning can be applied for recommending data preparation pipelines [5]. In other words, meta-learning looks at past situations of model's training and learns how to make predictions for the new scenario it is handling [6].

In this paper, we propose a method to automate the data preparation process using meta-learning techniques, that we named as MetaPrep. Our proposal consists of pre-training a meta-model capable of predicting the best pipeline for a previously unseen dataset using a large set of varied datasets.

The main contributions of our work are:

- MetaPrep is a novel meta-learning-based algorithm to automate the data preparation for ML. Although we found a similar idea in the literature [2], [7], our method is the first to recommend the entire data preparation pipeline based on meta-learning.
- MetaPrep can be easily used in other applications and adapted to work with many ML platforms with little effort.
- MetaPrep achieved accuracy rates similar to Learn2Clean [2], which is considered the state-of-the-art for automating data preparation, but with a significant reduced runtime.
- Since MetaPrep recommends a set of five data preparation pipelines, it is useful for data scientists with different degrees of experience. While the beginners may use the best-ranked pipeline as an out-of-the-box recommendation, more experienced researchers or practitioners may use their knowledge to choose one of the five recommendations.

II. DATA PREPARATION

Automating the data preparation process intends to allow developers and data scientists to reduce the amount of time consumed by this phase. As a side effect, they may spend this effort in configuring the ML algorithm or deploying the generated model.

The amount of techniques available for preprocessing data is proportional to its high variability, ranging from data of the most diverse types (numerical, categorical, dates and others), in different scales and even with low quality due to the presence of inconsistencies and missing values [8]. In this

scenario, human knowledge of the various techniques available for data preprocessing is usually necessary to choose which ones to apply to the data at hand.

The data preparation techniques used in the literature are commonly divided into three groups, namely: data integration, data cleaning and data transformation [2], [9].

A. Data integration

Frequent operations used to integrate data from several sources are: set unification, attribute correlation analysis, detection of equal values and detection of conflicts between data from different sources [8].

B. Data cleaning

The goal of data cleaning is to correct the original dataset, fixing some characteristics of the data. For example, some instances of the dataset do not have any given value for one or more attributes. This is a problem for statistical analysis and one of the main difficulties for training ML models. In this way, among its main techniques we can mention case deletion for the treatment of missing data, local outlier factor (LOF) for the detection of outliers and oversampling for class balancing. [8].

C. Data transformation

In general, since some ML algorithms are able to deal only with certain types of data (for example, numeric or symbolic values), sometimes it is necessary to convert the dataset to a specific format. The transformations applied to the data do not remove or generate new attributes from the original dataset, but transform the distribution of the original values into a new set of values with the desired properties [6]. For example, the Support Vector Machine (SVM) cannot handle categorical data, requiring a transformation to numerical data to apply the algorithm.

III. META-LEARNING

Meta-learning is an approach that investigates meta-data, which is the data that describes the structure of the main dataset, usually implicitly. Its main goal is to acquire knowledge (known as meta-knowledge) by mapping the characteristics of models trained in previous experiments. Meta-knowledge and conventional datasets share a similar attribute-value structure. However, while datasets are separated into attributes and targets, meta-knowledge is separated into meta-attributes and meta-targets. Meta-attributes are the information undeclared and not formally expressed in each dataset. It is implicit in each previously observed data, known as meta-examples, through descriptive characteristics like the number of examples, number of attributes, and statistics that describe the data distribution. The meta-targets describe the performances that the meta-examples obtained in past training. Usually, it is defined by the model with the best experimental result [5]. In summary, meta-learning analyzes past results to draw conclusions to be used as support for new problems.

Meta-attributes must be defined to describe the main properties of the dataset [10].

A. Statistical and information-theoretic characterization

One widely-used way to extract meta-attributes from the meta-examples consists of characterizing the data by statistical and information-theoretic measures, such as number of classes, attributes and samples, standard deviation, kurtosis, data skewness and mutual information [10]. A good set of meta-attributes in this category must be simple and efficiently calculated and needs to help determine performance of the algorithm used for training [11].

B. Model-based characterization

The meta-example characterization can also rely on the properties of ML models applied to its data [5]. Two advantages of this characterization method are: (1) as the characterization is not limited to the distribution of the data, the dataset can incorporate the complexity and assist in the performance of the hypothesis; (2) the resulting model can serve as a basis for explaining the reasons behind the performance of the ML algorithm [12].

C. Landmarking

Landmarking consists of characterizing the data through the performance of simple ML algorithms, generating meta-attributes with similar values when the performance is similar [12]. This technique calculates evaluation measures obtained by various machine learning algorithms, called landmarks, to characterize the data. Some examples of meta-attributes in this category are accuracy, F1-score, and area under the ROC curve.

IV. THE DATA PREPARATION PIPELINE RECOMMENDATION SYSTEM

We propose a data preparation pipeline recommendation method for using meta-learning, referred to as MetaPrep. This section describes how the proposed recommendation system was developed and the relationship between its steps.

A. Methodology

The developed code is open source², as well as the datasets are freely available. The experiments were performed on a remote server with two processors Intel Xeon E3-12xx v2 (Ivy Bridge), two 16GB RAM memory (32GB RAM in total) and 50GB internal storage.

1) *Data preparation techniques*: The first step in the process was the definition of the techniques used for data preparation. Thus, based on the techniques most commonly used in ML models, the following techniques were selected:

- **Imputation**: Deletion case, mean, median and most frequent;
- **Case deletion**: This method excludes samples with missing values. The case deletion needs to be used after extensive data analysis, as this technique may cause a significant loss of information [13].

²<https://github.com/fernandozagatti/metaprep>

- **Imputation mean:** This method consists of filling missing values of a given attribute (this being defined as the columns of the dataset) by the mean of the known values of this same attribute [14].
- **Imputation median:** This method works similarly to the previous one, with the missing data being filled with the general median or the median of all known values. [14].
- **Imputation most frequent:** This method will replace all missing values with the most frequent value in the column, and can be used both in numeric data and strings.
- **Categorical-numerical transformation:** one-hot-encoding and label encoder;
 - **Symbolic-numeric conversion:** Some algorithms cannot handle non-numeric data, such as neural networks and SVM. Two of the most famous techniques to convert symbolic into numeric data are: one-hot-encoding, which tries to transform each categorical data into a new column with 1 or 0, and label encoder, which transforms each categorical value into a specific number [6].
- **Standardization and normalization:** Standard Scaler, minmax and normalizer;
 - **Standard Scaler and minmax:** This methods aims to rescale the attributes, in such a way that the data average is equal to 0 and the standard deviation is equal to 1. Another standardization alternative is to rescale the values between a minimum and a maximum value (0 and 1, for example) [15].
 - **Normalizer:** This method consists of rescaling each of the samples in a unitary norm, considering that the techniques present in normalization commonly use the mathematical l1, l2 or maximum norms [15].
- **Class balancing:** Oversampling and undersampling.
 - **Random oversampling:** This method tries to repair the distribution of classes by randomly replicating samples in minority classes, thus being a non-heuristic technique. The problem with this method is precisely the replication, which can cause overfitting by generating several duplicated data [16].
 - **Random undersampling:** This is also a non-heuristic method, but unlike the previous one, this tries to balance classes by randomly excluding samples from the majority classes [16].

With the appropriate combinations considering all the techniques in these four groups, 180 objective pipelines were defined for data preparation. All pipelines can be viewed on Github³.

2) *Datasets collection:* All datasets were collected from OpenML [17], a collaborative platform that aims to share and develop ML research. The following filter was initially applied

to select the necessary datasets for the experiments: 1 to 10 classes and at least 501 instances.

The choice of the final datasets took into account some selection criteria such as having structured data and allowing the proper application of the selected techniques.

3) *Meta-targets:* Each meta-example passes through all pipelines, and the resulting cleaned data is used to determine the most effective pipeline. For this, we perform an experiment using Random Forests and ranking the pipelines by the achieved accuracy. The five top-ranked pipelines are used to establish the target of the respective meta-example.

4) *Meta-attributes:* For the extraction of meta-attributes, we used Pymfe [18], a package for extracting meta-features in Python. This tool allowed the extraction of 73 statistical and information theory-based meta-attributes. For this method, all possible meta-attributes that Pymfe could extract were used. A delimitation for meta-attributes is a proposal for future work.

5) *Meta-models:* We created the meta-models using kNN on the meta-knowledge to identify the most similar datasets according to their characteristics. Specifically, we used the 1NN algorithm to avoid interferences of the hyperparameter of the meta-model in our conclusions for this first experiment. Moreover, the 1NN is intuitive, and we do not need to define a strategy to aggregate the pipelines recommended for more than one meta-example.

In the single-model MetaPrep, all the meta-examples are used in the input dataset for the meta-model. On the other hand, the multiple-model MetaPrep uses disjunct subsets of the meta-knowledge, creating separate models.

B. MetaPrep composition

The recommendation system was developed based on experiments carried out with 80 datasets, also called meta-examples (illustrated as the step 1 in Figure 1), and the combination of 11 data preparation techniques, totaling 180 possible pipelines. Hence, to create the meta-model for recommending data preparation pipelines, the system performed two auxiliary tasks: (i) characterizing the meta-example; and (ii) training the models to determine the best data preparation pipeline for each case.

Firstly (from number 2 to 4 in Figure 1), the 180 pipelines were applied to each of the 80 meta-examples. After performing all the data preparation techniques specified in a pipeline, a RandomForest model was trained on the refined dataset and tested using a holdout procedure. The five pipelines with the best model's accuracy become the meta-targets for each meta-example.

Then (from number 3 to 4 in Figure 1), we extracted the characteristics of each meta-example. These characteristics, called meta-attributes, are information that are implicitly presented in each dataset, such as the number of unique class labels, examples, and attributes, and statistics from each attribute, such as standard deviation, kurtosis, among others. In total, we used 73 meta-attributes to describe each meta-example.

From the collection of meta-attributes and meta-targets, the meta-knowledge (illustrated as the step 4 in Figure 1) was

³<https://github.com/fernandozagati/metaprep/blob/master/pipelines/All%20pipelines.txt>

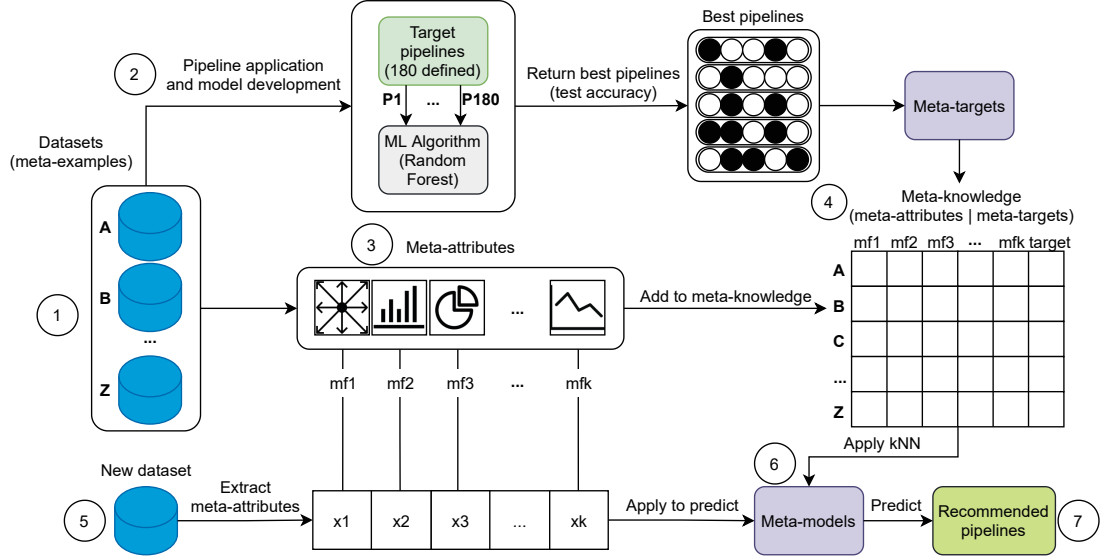


Fig. 1. Our proposal: a meta-learning-based recommender system to predict best pipelines for data preparation.

created as the basis for the meta-models that are the core of our pipeline recommendation system for data preparation. The creation of the meta-models was done through kNN to identify the meta-examples closest to a new entry.

We observe that this procedure may lead to identical results of several pipelines depending on each dataset's characteristics. For instance, if a dataset has no categorical attributes, all the pipelines that include categorical-numerical transformations have the same outcome among them and pipelines with the same techniques but this specific transformation. For instance, the pipelines Standard Scaler -> Oversampling and One-hot-encoding -> Standard Scaler -> Oversampling are equivalent for datasets with no categorical values.

To reduce this issue's influence, we propose a variation of our method, including a pipeline subset selection step. The advantages of avoiding this kind of replication are twofold: (i) it reduces the number of classes the meta-model needs to predict without losing relevant pipelines; and (ii) it avoids the recommendation of a pipeline missing essential techniques. For instance, if a dataset contains missing values, we use a meta-model that assuredly recommends an imputation technique. Otherwise, the meta-model may recommend a pipeline that ignores this issue. Alternatively, if the dataset does not contain missing values, we apply a meta-model trained to predict pipelines that do not include imputation techniques.

To define the subset of pipelines, we grouped the datasets into five categories, considering the characteristics of the datasets. Consequently, we create five meta-models. The characteristics used to separate the datasets are the following:

- 1) **Dataset without categorical data and missing values**: no imputation or transformation is required.
- 2) **Dataset without categorical data but with missing values**: imputation of missing values is necessary, transformation techniques are discarded.

- 3) **Dataset without missing values but with categorical data**: only the transformation of categorical values is necessary, imputation is discarded.
- 4) **Dataset with categorical data and missing values**: this case takes both imputation and transformation, and since there are nulls in the categorical data, the imputation needs to be compatible.
- 5) **Dataset with categorical data but with missing values only in numeric columns**: this case takes both imputation and transformation, and since there are nulls only in the numeric data, it can be any type of imputation.

Considering these groups, we propose and compare the MetaPrep in two versions: single and multiple meta-models. Figure 2 illustrates these variations. In the single meta-model version, when the recommendation system receives a new dataset for automatic preparation, it is characterized, compared with the meta-knowledge (created using all the meta-examples). The data preparation pipelines are suggested based on the meta-targets of the most similar meta-example. The multiple meta-models version works as following. The recommendation system starts by extracting the meta-features to characterize the new dataset. Then, it checks which of the five meta-models (developed for each previously mentioned group) fit the new dataset better. Finally, instead of comparing the new entry with the entire meta-knowledge base, it is compared only with the same group's meta-examples.

C. Prediction of a new case

Once a new dataset is presented to the system, it needs to recommend a data preparation to apply to it. The new input dataset passes through the meta-attributes extraction and is used as input by the meta-models.

The extraction of meta-attributes has a special parameter called "transformer"; by default the algorithm uses "gray",

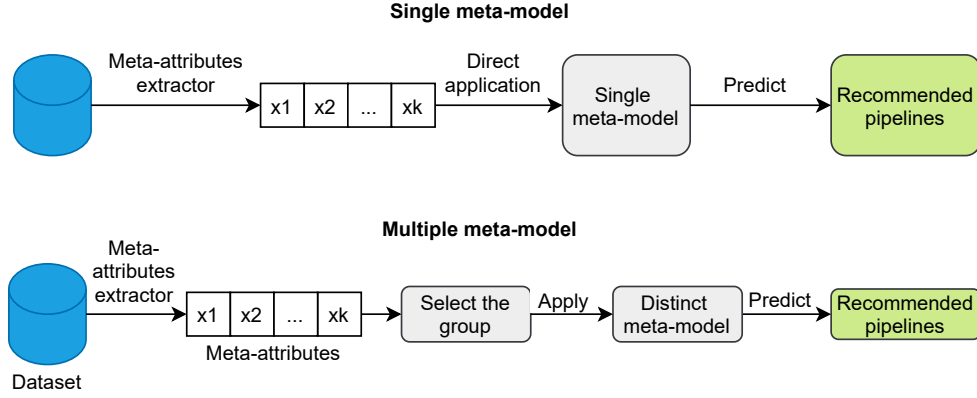


Fig. 2. Difference between single (top) and multiple meta-models (bottom) MetaPrep

where all categorical-type data will be binarized with a model matrix strategy. However, it is possible to use "None", where the categorical attributes are not transformed and are ignored during characterization. Although the default method is more assertive because it can characterize the entire dataset without losing information, a lot of RAM is consumed in the process. For cases of low available memory, the second method is a viable alternative.

After extracting meta-attributes, there is an intermediate step to choose which meta-model will be used if we apply the multiple meta-models approach. The meta-models then predict the meta-examples that most resemble this new entry and recommend a list of five pipelines for this new dataset. We consider that, for an inexperienced user, choosing the top-ranked recommendation is a proper procedure. We used this assumption to evaluate our proposal.

V. EXPERIMENTAL EVALUATION

A. Results

We trained models after each of the 180 pipelines on each of the 80 datasets to create the meta-data. Then, the five models with the best accuracy comprise the meta-targets. In this way, the recommendation system has a total of 400 possible recommendations.

1) *Results on open datasets:* We experimented with four different datasets extracted from Kaggle⁴, an online platform for data scientists to publish datasets and create and explore ML models. The selected sets were:

- Titanic - Machine Learning from Disaster⁵: Contains categorical data, numerical data and missing values.
- Tic-Tac-Toe Endgame Data Set⁶: Contains only categorical data.
- Bike Buyers 1000⁷: Contains categorical and numerical data.

⁴<https://www.kaggle.com/>

⁵<https://www.kaggle.com/c/titanic>

⁶<https://www.kaggle.com/rsrishav/tictactoe-endgame-data-set>

⁷<https://www.kaggle.com/heeraldedhia/bike-buyers>

- Cardiovascular Disease dataset⁸: Contains only numerical data.

To compare the impact of different data preparation pipelines, we performed three types of comparisons: (1) the direct application of datasets in an AutoML platform (in this case, Auto-sklearn was chosen), (2) the application of Learn2Clean, and (3) the application of MetaPrep (in its two versions).

Our approaches obtained very similar results. However, as can be seen in Table I, the multiple meta-models approach led to better recommendations. On the other hand, recommending a single meta-model often confused what it should recommend. These cases could be checked in the tests, in which the single meta-model did not recommend imputations to the Titanic dataset and recommended unnecessary transformations in the Cardiovascular Disease dataset.

Both Auto-sklearn, which uses a default preprocessing (Imputation by Mean, One-Hot-Encoding, and Standard Scaler), and the Learn2Clean, which uses reinforcement learning, had difficulty handling some datasets. Auto-sklearn requires much processing to carry out its training, and its default preprocessing may not be ideal, while Learn2Clean cannot handle categorical data, failing to recommend a pipeline for the dataset with only categorical attributes (Tic-Tac-Toe).

Table II brings the accuracy and execution time for each experiment, now evaluating an end-to-end pipeline: cleaning the data, applying ML, and evaluating the resulting model.

Auto-sklearn could only perform training on the Cardiovascular Disease dataset, as the other datasets required manual adjustments before being applied in AutoML. Besides, Learn2Clean drops categorical columns, dealing only with numerical values in its training. That is the reason why Learn2Clean was unable to train the Tic-Tac-Toe dataset.

In contrast, the proposed approaches performed quickly, managing to recommend pipelines in seconds. Comparing the obtained accuracy, we note that both single and multiple meta-model versions achieved identical rates in three datasets.

⁸<https://www.kaggle.com/sulianova/cardiovascular-disease-dataset>

TABLE I
PIPELINES USED BY EACH TOOL

Titanic - Machine Learning from Disaster	
Autosklearn	Default preprocessor
Learn2Clean	Local Outlier Factor
Single meta-model	Label Encoder -> Standard Scaler -> Oversampling
Multiple meta-models	Case deletion ->Label Encoder -> Standard Scaler ->Oversampling
Tic-Tac-Toe Endgame Data Set	
Autosklearn	Default preprocessor
Learn2Clean	N/A
Single meta-model	Imputation mean ->One-hot-encoding -> Standard Scaler ->Oversampling
Multiple meta-models	Label Encoder ->Normalizer -> Oversampling
Bike Buyers 1000	
Autosklearn	Default preprocessor
Learn2Clean	Z-score-based method ->Exact duplicate
Single meta-model	One-hot-encoding ->Standard Scaler -> Oversampling
Multiple meta-models	One-hot-encoding ->Standard Scaler -> Oversampling
Cardiovascular Disease dataset	
Auto-sklearn	Default preprocessor
Learn2Clean	Local Outlier Factor
Single meta-model	One-hot-encoding ->Standard Scaler -> Oversampling
Multiple meta-models	Standard Scaler ->Oversampling

TABLE II
RESULTS OF THE END-TO-END EVALUATION

		Total time	Accuracy
Autosklearn	Titanic	1800s	78.73%
	Tic-Tac-Toe	1800s	100%
	Bike Buyers	1800s	68.66%
	Cardio	1800s	73.56%
Learn2Clean	Titanic	405s	72.04%
	Tic-Tac-Toe	N/A	N/A
	Bike Buyers	539s	68.17%
	Cardio	>1800s	73.25%
Single meta-model	Titanic	N/A	N/A
	Tic-Tac-Toe	2.3s	98.07%
	Bike Buyers	3.9s	71.72%
	Cardio	17.9s	72.88%
Multiple meta-models	Titanic	7.8s	84.15%
	Tic-Tac-Toe	2.3s	98.07%
	Bike Buyers	3.9s	71.72%
	Cardio	17.9s	72.88%

However, the single meta-model had difficulty identifying the techniques that should be used in the Titanic dataset.

The experiment demonstrated the efficiency of meta-learning for recommending pipelines. Notably, the approach using multiple meta-models could recommend adequate techniques in a short runtime.

VI. CONCLUSION

This work addressed a current practical need for automating the data preparation on AutoML platforms using meta-learning. In this context, the preliminary results showed the applicability of meta-learning for selecting data preparation techniques, managing to recommend pipelines efficiently, and having the possibility of being customized by the developer.

As future work, we intend to perform a more in-depth analysis of the aspects of MetaPrep. In addition to evaluating other machine learning algorithms to perform the recommendation, we will investigate how the choice between the recommended pipelines may consider other evaluation metrics, aiming at pipelines with the potential to lead to models with better generalization.

REFERENCES

- [1] N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data management challenges in production machine learning," in *Proceedings of the 2017 ACM International Conference on Management of Data*, ser. SIGMOD '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 1723–1726. [Online]. Available: <https://doi.org/10.1145/3035918.3054782>
- [2] L. Berti-Equille, "Learn2clean: Optimizing the sequence of tasks for web data preparation," in *The World Wide Web Conference*. ACM, 2019, pp. 2580–2586.
- [3] M. Feurer, A. Klein, K. Eggenberger, J. Springenberg, M. Blum, and F. Hutter, "Efficient and robust automated machine learning," in *Advances in neural information processing systems*, 2015, pp. 2962–2970.
- [4] T. T. Le, W. Fu, and J. H. Moore, "Scaling tree-based automated machine learning to biomedical big data with a feature set selector," *Bioinformatics*, vol. 36, no. 1, pp. 250–256, 2020.
- [5] P. Brazdil, C. G. Carrier, C. Soares, and R. Vilalta, *Metalearning: Applications to data mining*. Springer Science & Business Media, 2008.
- [6] K. Faceli, A. C. Lorena, J. Gama, and A. C. P. d. L. F. d. Carvalho, *Inteligência artificial: uma abordagem de aprendizado de máquina*. LTC, 2011.
- [7] S. Krishnan, M. J. Franklin, K. Goldberg, J. Wang, and E. Wu, "Activeclean: An interactive data cleaning framework for modern machine learning," in *Proceedings of the 2016 International Conference on Management of Data*, 2016, pp. 2117–2120.
- [8] S. García, J. Luengo, and F. Herrera, *Data preprocessing in data mining*. Springer, 2015.
- [9] G. E. d. A. P. A. Batista, "Pré-processamento de dados em aprendizado de máquina supervisionado," Ph.D. dissertation, Universidade de São Paulo, 2003.
- [10] C. Castiello, G. Castellano, and A. M. Fanelli, "Meta-data: Characterization of input features for meta-learning," in *International Conference on Modeling Decisions for Artificial Intelligence*. Springer, 2005, pp. 457–468.
- [11] C. Castiello and A. M. Fanelli, "Computational intelligence for meta-learning: A promising avenue of research," in *Meta-Learning in Computational Intelligence*. Springer, 2011, pp. 157–177.
- [12] R. Vilalta, C. Giraud-Carrier, and P. Brazdil, "Meta-learning-concepts and techniques," in *Data mining and knowledge discovery handbook*. Springer, 2009, pp. 717–731.
- [13] R. J. Little and D. B. Rubin, *Statistical analysis with missing data*. John Wiley & Sons, 2019, vol. 793.
- [14] D. Banks, L. House, F. R. McMorris, P. Arabie, and W. A. Gaul, *Classification, Clustering, and Data Mining Applications: Proceedings of the Meeting of the International Federation of Classification Societies (IFCS), Illinois Institute of Technology, Chicago, 15–18 July 2004*. Springer Science & Business Media, 2011.
- [15] E. Bisong, "Introduction to scikit-learn," in *Building Machine Learning and Deep Learning Models on Google Cloud Platform*. Springer, 2019, pp. 215–229.
- [16] G. E. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD explorations newsletter*, vol. 6, no. 1, pp. 20–29, 2004.
- [17] M. Feurer, J. N. van Rijn, A. Kadra, P. Gijssbers, N. Mallik, S. Ravi, A. Müller, J. Vanschoren, and F. Hutter, "Openml-python: an extensible python api for openml," *arXiv:1911.02490*, 2019.
- [18] E. Alcobaça, F. Siqueira, A. Rivolli, L. P. F. Garcia, J. T. Oliva, and A. C. P. L. F. de Carvalho, "Mfe: Towards reproducible meta-feature extraction," *Journal of Machine Learning Research*, vol. 21, no. 111, pp. 1–5, 2020. [Online]. Available: <http://jmlr.org/papers/v21/19-348.html>